Expense Tracker

DevOps Capstone Project

Raaghav Rathi
24bcs10284

What I Built

A simple web app to track daily spending.

Add, edit and delete expenses with a title, amount, category and date.

See the total spent and a breakdown per category.

The app is small on purpose. The focus of the project is the DevOps workflow around it: testing, containers, CI/CD, security scanning, infrastructure as code, Kubernetes, GitOps and monitoring.

Three tiers

Frontend: React app served by nginx

Backend: FastAPI REST API

Database: PostgreSQL

Endpoints: /health, /ready, /metrics, CRUD on /api/expenses, /api/expenses/summary

Tech Stack

Application

Python, FastAPI
SQLAlchemy, Alembic
React, Vite
PostgreSQL 16

Containers

Docker (non-root images)
Multi-stage frontend build
Docker Compose

CI/CD + Security

GitHub Actions
GitHub Container Registry
Trivy image scanning

Infrastructure

Terraform
AWS VPC
AWS EKS

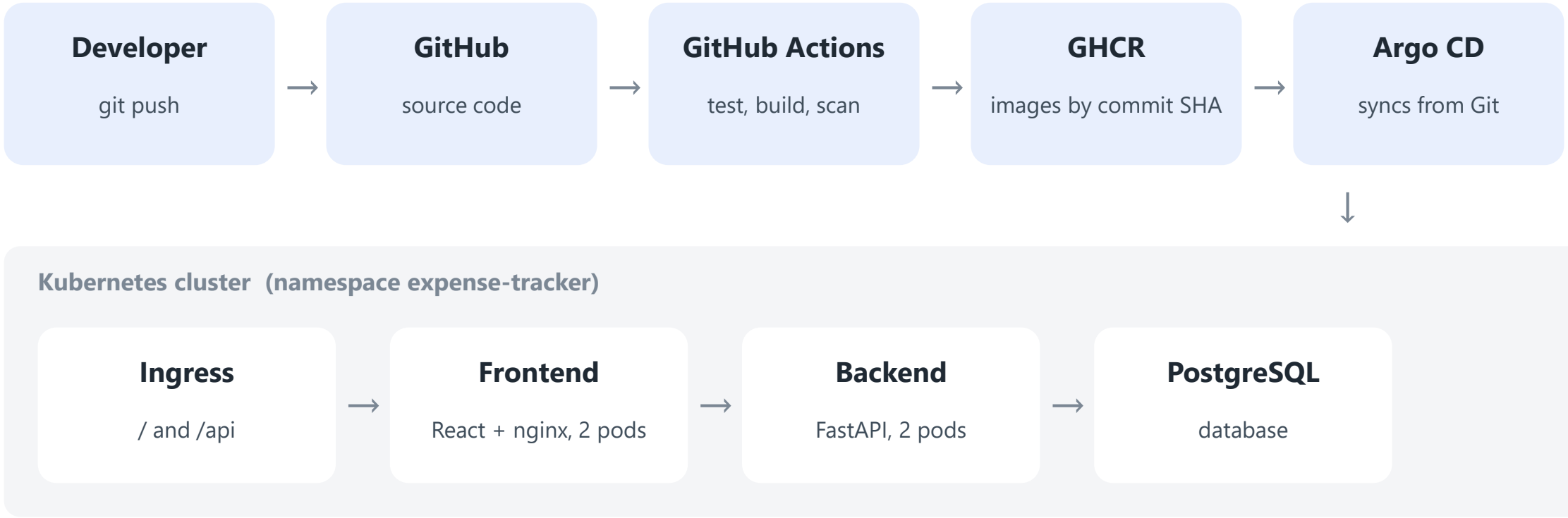
Kubernetes

Helm chart
ingress-nginx
Argo CD (GitOps)

Observability

Prometheus
Grafana
kube-prometheus-stack

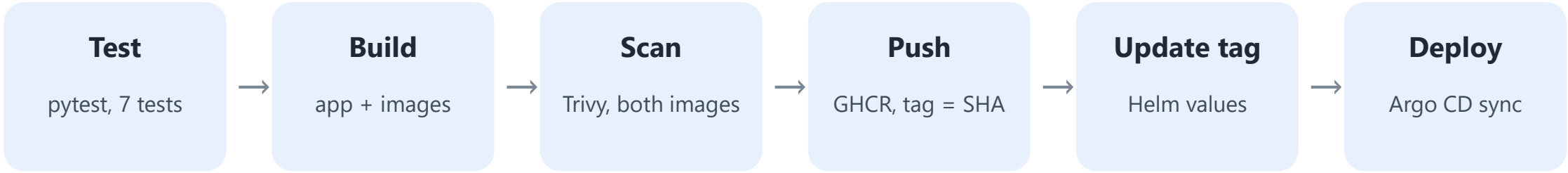
Architecture



Prometheus scrapes backend /metrics, Grafana shows the dashboard. Terraform creates the AWS VPC and EKS cluster.

CI/CD Pipeline

GitHub Actions runs on every push to main



Quality gates

A failing test stops the pipeline.
A HIGH or CRITICAL vulnerability stops the pipeline.
Nothing broken or vulnerable is pushed.

Why tag by commit SHA

Every image points to the exact code it came from.
Easy to see what is running and to roll back.

DevSecOps: Trivy Scan

Trivy scans the OS packages and libraries in both images. The job fails on HIGH or CRITICAL issues that have a fix.

Found while building

Backend: starlette 0.41.3

CVE-2025-62727 (HIGH): denial of service through many overlapping Range headers.

Frontend: nginx 1.27 alpine base

CVE-2026-31789 (CRITICAL): OpenSSL heap buffer overflow, plus 40 more.

Fixed

Upgraded FastAPI to 0.143.0 and the Prometheus instrumentator to 8.1.0, so Starlette moved to 1.8.0.

Moved the frontend base image to nginx 1.30 alpine.

Result: both images scan clean and the pipeline passes.

Infrastructure with Terraform

What Terraform creates on AWS

Region ap-south-1 (Mumbai)

VPC 10.30.0.0/16

2 public subnets in 2 availability zones

EKS cluster, Kubernetes 1.31

Managed node group, t3.medium workers

46 resources in total

Workflow

terraform init

terraform plan

terraform apply

aws eks update-kubeconfig

terraform destroy after the demo to avoid cost

No credentials in the repo, only terraform.tfvars.example

Kubernetes, Helm and GitOps

Helm chart

Backend: 2 replicas

Frontend: 2 replicas

PostgreSQL

ClusterIP services

Readiness and liveness probes

Ingress

/ goes to the frontend

/api goes to the backend

Deployed with:

helm upgrade --install

Argo CD

Watches the Helm chart in GitHub

Auto sync and self heal

The pipeline updates the image tag in Git, Argo CD deploys it

Git is the single source of truth

Observability

Backend
/metrics

→

ServiceMonitor
tells Prometheus what to scrape

→

Prometheus
collects every 15s

→

Grafana
dashboard

Expense Tracker dashboard in Grafana

Requests per second for each endpoint
p95 latency
Error rate (5xx responses)
Number of backend targets up

Key Learnings

Gates save time

Tests and Trivy scans in the pipeline catch problems before anything is deployed.

Images by commit SHA

Each deployed image maps to exact code, which makes debugging and rollbacks simple.

GitOps

With Argo CD, Git decides what runs. A push is enough to deploy.

Real bug found

Two backend pods ran database migrations at the same time. A PostgreSQL advisory lock fixed it.